SYSTEM PROGRESS REPORT

[SYSTEM TITLE]

Proponents	**[NAMES]**
Adviser	**[ADVISER NAME]**
School / Program	**[SCHOOL, PROGRAM]**
Report date	**[DATE]**

Status as of 14 November 2026, based on the source code in the project repository (branch *main*, commit 80a7a81).

1. System Overview

The system is a web-based self-service ordering kiosk with integrated real-time stock inventory and transaction management for Zak's Sizzling Hub, a sizzling-plate restaurant. Customers order on an in-store touchscreen kiosk (a Raspberry Pi with a 1024 × 600 display) or on their own phones by scanning a QR code. They browse the menu by category, search or filter by price, add dishes with special instructions, choose dine in or take out, and pay at the counter (online payment through QR Ph is implemented but not yet live). Each order receives a unique order number and an estimated preparation time. On the in-store kiosk the receipt is printed on a thermal printer, and a second printer in the kitchen prints a ticket for each paid order.

Staff use a back office on a separate address. At the counter, a light point of sale takes payment for kiosk orders (cash with the change worked out, or GCash or Maya recorded with its reference number) and prints a receipt, rings up walk-in customers' orders, and cancels paid orders with a reason. Staff follow orders on a kitchen display that updates in real time and adjust stock with a recorded reason. Every dish is made from a bill of materials: when an order is paid the system deducts its raw materials (items sold as they are, such as bottled drinks, by the piece), and every open kiosk updates its sold-out items without refreshing. The Administrator additionally manages products, recipes, prices, costs and photos, imports products and recipes in bulk from CSV or Excel files, views sales analytics with CSV, Excel and PDF export, manages user accounts and roles, and prints the kiosk QR code.

A background worker checks stock levels every hour, sends low-stock e-mail alerts when e-mail is configured, and backs up the database every night to off-site cloud storage. The system is deployed on Railway as separate customer and staff web addresses, an API service, a worker and a managed PostgreSQL database. This report describes only what is present in the source code; every status in Section 4 cites the files that implement it.

Users and roles

Role	What the role can do in the current code
Customer (no sign-in)	Order at the kiosk or on a phone: browse, search, filter by price, add dishes with instructions, choose dine in or take out, pay at the counter, and see (and on the kiosk, print) the receipt.
Staff	Dashboard without cost figures, raw-material and ready-made stock adjustments and the stock movements log, kitchen display with printed kitchen tickets, and the counter's point of sale: taking payment, ringing up walk-in orders, and cancelling orders.
Administrator	Everything Staff can do, plus product management (prices, costs, photos, recipes), bulk import, raw-material set-up, analytics and exports, user and role management, and the kiosk QR code.

Access is enforced by the API on every route (`apps/api/src/middleware/authorize.ts`); the web pages apply the same rules so that users only see what they may use. Figure 1 on the next page shows how the users and the parts of the system work together.

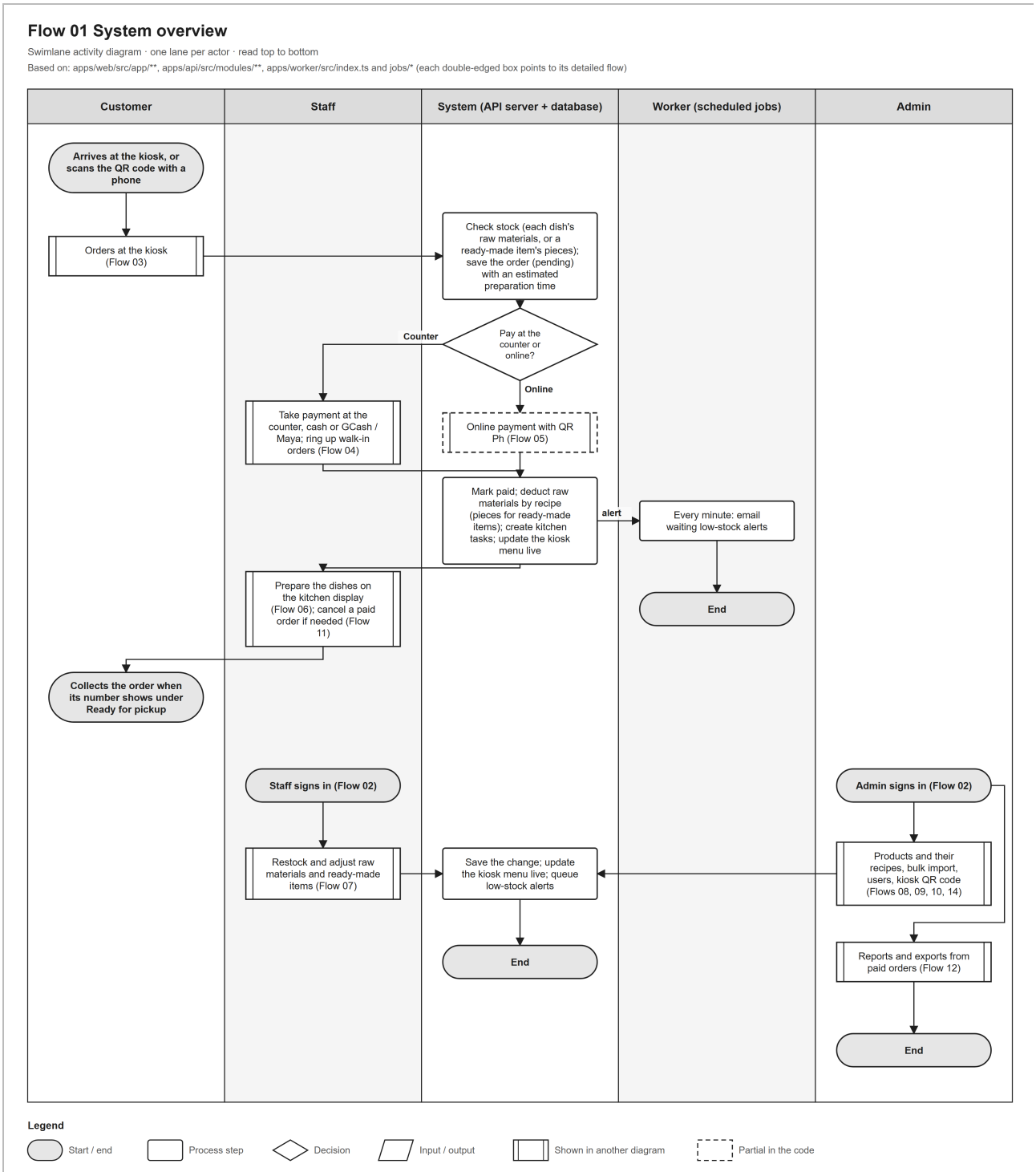


Figure 1. System overview: swimlane activity diagram of the main processes.

2. Technology Stack

Layer	Technology	Use in the system
Language	TypeScript	All application code; one repository with npm workspaces.
Web application	Next.js 16, React 19, Tailwind CSS 4	Customer kiosk, back office and kitchen display (`apps/web`).
API server	Node.js, Express 4, Zod	REST API with validation and role checks (`apps/api`).
Real-time updates	Socket.IO	Stock, new-order and kitchen status events.
Database	PostgreSQL 18, Prisma 6	Schema, migrations and queries (`apps/api/prisma`).
Authentication	JSON Web Tokens, bcrypt	Staff sign-in, anonymous kiosk sessions, password hashing.
Background jobs	node-cron, Brevo / Resend / Nodemailer, pg_dump	Low-stock checks and e-mails, nightly backups (`apps/worker`).
File handling	ExcelJS, PDFKit, qrcode	Excel import and export, PDF reports, kiosk QR code.
Payments	PayMongo	QR Ph online payment, scanned with any bank or e-wallet app (implemented, not yet live).
Storage	Cloudflare R2	Off-site database backups.
Hosting	Railway	Customer and staff web addresses, API, worker, PostgreSQL.
Kiosk and kitchen hardware	Raspberry Pi, 1024 × 600 touchscreen, two RP21UB-Y 58 mm thermal printers	In-store kiosk and kitchen; a print service (Node.js, `print-service`) on the device next to each printer prints the customer receipt and the kitchen ticket.
Testing	Vitest, Supertest	Automated unit, integration and security tests (`apps/api/test`).

3. Overall Progress Summary

The features assessed are those described in the manuscript, grouped into sixteen modules as in the progress report of 25 September 2026, together with two planned activities identified by the proponents (testing PayMongo with real keys and structured testing on the physical kiosk), two features added by the proponents (kitchen ticket printing, and removing a raw material) and two added from the panel's review in November 2026 (walk-in orders at the counter, and searching and categorising raw materials). Four offline and Progressive Web App features were removed from the manuscript by the adviser's decision, and the two barcode features by the panel's; they are listed but not scored.

Status	Features	Meaning
Completed	75	Works end to end in the code (interface, logic and data saving where relevant).
Partial	3	Started; the missing parts are stated.
Not yet started	4	No implementation in the code.
Removed from scope	6	Removed by the adviser's or the panel's decision; not scored.
Total	**88**	**82 scored features**

No feature was found to be a placeholder, that is, a screen or route without real logic behind it.

Overall progress: 93.3%. Each scored feature counts 1 point if Completed, ½ point if Partial and 0 if Not yet started. Overall progress = (75 × 1 + 3 × ½) ÷ 82 = 76.5 ÷ 82 = 93.3%. Features removed from scope are excluded from both the points and the total. Every feature has the same weight, so the figure shows how much of the described functionality exists, not how much effort remains.

Module	Completed	Partial	Not yet started	Removed	Progress
1. Kiosk ordering interface	9	1	0	0	95%
2. Checkout and payments	5	1	2	0	69%
3. Front-desk order handling	6	0	0	0	100%
4. Kitchen display system	5	0	0	0	100%
5. Real-time inventory synchronisation	3	0	0	0	100%
6. Product and catalogue management	5	1	0	1	92%
7. Inventory monitoring and stock adjustments	8	0	0	0	100%
8. Analytics, reports and data export	7	0	0	0	100%
9. Authentication, roles and user management	5	0	0	0	100%
10. Security measures	8	0	0	0	100%
11. Background worker and backups	3	0	0	1	100%
12. Offline support (PWA)	0	0	0	3	—
13. Hardware integration	1	0	0	1	100%
14. QR code and multi-device access	2	0	0	0	100%
15. Deployment	3	0	0	0	100%
16. Testing and quality assurance	5	0	2	0	71%

Module progress uses the same scoring as the overall figure. The progress report of 25 September 2026 averaged the sixteen module percentages and included the offline features; the two figures are therefore not directly comparable.

4. Module and Feature Status

Every feature is listed with its status, a short remark, and the main files that implement it (paths relative to the repository root). A feature is marked Completed only when the interface, the logic and, where relevant, the saving of data are all present in these files.

Feature	Status	Remarks	Main files
1. Kiosk ordering interface			
Product catalogue with names, prices and availability	**Completed**	One category at a time as a grid of tiles; sold-out dishes are dimmed and listed last.	`apps/web/src/components/kiosk/MenuGrid.tsx` `apps/web/src/components/kiosk/ItemTile.tsx` `apps/api/src/modules/products/routes.ts`
Live availability updates	**Completed**	Sold-out status changes on every open kiosk without a page refresh (inventory:updated event).	`apps/web/src/hooks/useCatalog.ts` `apps/api/src/modules/inventory/fulfillment.ts`
Item details: description, ingredients, allergens, nutrition	**Completed**	Set in the product form: ingredients, an allergen checklist with suggestions, and nutrition per serving looked up in USDA FoodData Central or calculated from the recipe; the kiosk labels the values as estimates.	`apps/web/src/components/kiosk/ProductModal.tsx` `apps/web/src/components/admin/ProductDetailsFields.tsx` `apps/api/src/modules/nutrition/calc.ts`
Category navigation	**Completed**	Category rail with a picture tile per category.	`apps/web/src/components/kiosk/CategoryRail.tsx`
Keyword search	**Completed**	Searches dish names across all categories.	`apps/web/src/components/kiosk/TopBar.tsx` `apps/web/src/app/page.tsx`
Order (cart): add, change quantity, remove, running total	**Completed**	Saved on the device so a page refresh keeps the order.	`apps/web/src/hooks/useCart.tsx` `apps/web/src/components/kiosk/OrderBar.tsx` `apps/web/src/components/kiosk/CartDrawer.tsx`
Special instructions sent to the kitchen	**Completed**	Each dish's own quick options, set in the product form, which can also switch them off for a dish (panel feedback); plus a typed note, limited to 140 characters.	`apps/web/src/components/kiosk/ProductModal.tsx` `apps/web/src/components/admin/ProductForm.tsx` `apps/web/src/app/kitchen/page.tsx`
Kiosk session and idle-timeout reset	**Completed**	Warning after 60 s without activity; the order is cleared 20 s later.	`apps/web/src/hooks/useIdleTimer.ts` `apps/web/src/components/kiosk/IdleTimeoutOverlay.tsx`
Product images	**Completed**	Uploaded by the Administrator, reduced in size in the browser, stored in the database.	`apps/web/src/components/admin/ProductForm.tsx` `apps/web/src/lib/imageResize.ts` `apps/api/src/modules/products/image.ts`
Filter by price range or popularity	**Partial**	Price-range filter implemented; no popularity filter.	`apps/web/src/components/kiosk/MenuGrid.tsx` `apps/web/src/lib/priceRanges.ts`

Feature	Status	Remarks	Main files
2. Checkout and payments			
Payment method selection (counter, QR Ph)	**Completed**	QR Ph is shown as unavailable until online payment is configured.	`apps/web/src/app/checkout/page.tsx` `apps/api/src/modules/payments/routes.ts`
Pay-at-counter order placement	**Completed**	The server re-checks availability and stock before accepting the order.	`apps/web/src/app/checkout/page.tsx` `apps/api/src/modules/orders/service.ts`

Online payment through PayMongo (QR Ph)	**Partial**	QR Ph is the one online method; it replaced GCash and Maya on 26 October 2026. The kiosk shows the QR Ph code on its own screen, checks for the payment every 3 seconds, and also accepts PayMongo's signed webhook; covered by automated tests with PayMongo simulated. Not yet run with PayMongo keys; production has none yet.	`apps/api/src/modules/payments/service.ts`
			`apps/api/src/lib/paymongo.ts`
			`apps/web/src/components/kiosk/QrPaymentScreen.tsx`

Digital receipt with a unique order number	**Completed**	Order number, order type, items, total, payment method and estimated preparation time.	`apps/web/src/components/kiosk/ReceiptModal.tsx` `apps/api/src/modules/orders/service.ts`
Estimated preparation time	**Completed**	Longest dish + 1 min per extra serving + 2 min per order already in the kitchen; shown at checkout and on the receipt.	`apps/api/src/modules/orders/prepTime.ts` `apps/web/src/app/checkout/page.tsx`

Receipt printing	**Completed**	The in-store kiosk prints the customer's receipt (restaurant name, order number, order type, items with prices, total, paid or pay at the counter, ready time) on its 58 mm thermal printer through the print service on the Raspberry Pi. Verified on the physical kiosk on 27 October 2026.	`apps/web/src/lib/receiptPrinter.ts`
			`apps/web/src/app/checkout/page.tsx`
			`print-service/server.js`

QR-code payment option (manuscript wireframe)	**Not yet started**	Not present in the code.	—
Testing of PayMongo with sandbox and live keys	**Not yet started**	Planned. Requires PayMongo keys and the client's approval.	—
3. Front-desk order handling			
List of orders awaiting payment	**Completed**	Unpaid orders and paid orders flagged with a stock issue; updates live when an order is placed, paid, edited or cancelled.	`apps/web/src/app/staff/orders/page.tsx` `apps/api/src/modules/orders/service.ts`

Feature	Status	Remarks	Main files
Take payment at the counter	**Completed**	A light point of sale (panel feedback): cash with the cash received and the change, or GCash or Maya recorded with its reference number (usable once); then an on-screen receipt printed on the counter computer's printer. Marks the order paid, deducts stock and sends it to the kitchen in one transaction; a stock shortage rolls everything back.	`apps/web/src/components/staff/PaymentPanel.tsx` `apps/web/src/components/staff/CounterReceipt.tsx` `apps/api/src/modules/payments/service.ts`
Walk-in orders at the counter	**Completed**	Added from the panel's review: staff ring up a customer who doesn't use the kiosk (menu grid with search and categories, sold-out dishes greyed out, each dish's instructions, dine in or take out), then take payment.	`apps/web/src/components/staff/PosNewOrder.tsx` `apps/api/src/modules/orders/routes.ts`
Look up an order by its number	**Completed**	Typed search field (not a keypad).	`apps/web/src/app/staff/orders/page.tsx`
Transaction history screen	**Completed**	Administrator only: orders by date with status, payment and order-number filters, totals with the average order value, and each order's timeline.	`apps/web/src/app/admin/transactions/page.tsx` `apps/api/src/modules/orders/service.ts`
Order status progression and cancellation	**Completed**	Confirmed, then preparing and ready as the kitchen works, then completed when the kitchen presses Handed over; unpaid orders can be cancelled in the payment queue, and paid ones in its Paid orders tab until handed over (no longer on the kitchen display, per the panel), with a reason, unstarted dishes returned to stock and the refund recorded by staff. A status board on the counter screen is the ready notice (not on a phone receipt; see 7.1).	`apps/api/src/modules/kitchen/service.ts` `apps/web/src/app/board/page.tsx` `apps/web/src/components/staff/PaidOrders.tsx`
4. Kitchen display system			
Real-time order queue (number, time, items, instructions)	**Completed**	Oldest first; each dish has its own status.	`apps/web/src/app/kitchen/page.tsx` `apps/api/src/modules/kitchen/service.ts`
WebSocket push without polling	**Completed**	order:created and kitchen:task_updated events.	`apps/api/src/websocket/index.ts` `apps/web/src/lib/websocket.ts`
Status updates with start and completion times	**Completed**	Pending, In progress, Completed per dish.	`apps/api/src/modules/kitchen/service.ts`
Dine-in / take-out indicator	**Completed**	Chosen by the customer after the welcome screen; take-out orders are highlighted.	`apps/web/src/app/kitchen/page.tsx` `apps/web/src/components/kiosk/OrderTypeScreen.tsx`

Feature	Status	Remarks	Main files
Kitchen ticket printing	**Completed**	A second thermal printer at the kitchen display prints a ticket with the order number and dishes (with special instructions) when a paid order reaches the kitchen; each order prints once and a failed ticket is retried. Verified on 27 October 2026.	`apps/web/src/app/kitchen/page.tsx` `apps/web/src/lib/receiptPrinter.ts` `print-service/server.js`
5. Real-time inventory synchronisation			
Automatic stock deduction when an order is paid	**Completed**	Atomic deduction; stock never goes below zero.	`apps/api/src/modules/inventory/fulfillment.ts`
Live stock broadcast to kiosks and back office	**Completed**	inventory:updated event after every change.	`apps/api/src/modules/inventory/fulfillment.ts` `apps/api/src/modules/inventory/service.ts`
Raw-material (ingredient) deduction by recipe	**Completed**	Every dish is made from a bill of materials (panel feedback): a recipe is required, and servings available are computed from raw-material stock. Items sold as they are, in the built-in Ready-made category, are counted in pieces.	`apps/api/src/modules/raw-materials/service.ts` `apps/api/src/modules/inventory/fulfillment.ts` `apps/web/src/app/admin/raw-materials/page.tsx`
6. Product and catalogue management			
Create, edit and delete products	**Completed**	Field validation; deletion is refused for products with order history.	`apps/web/src/app/admin/products/page.tsx` `apps/web/src/components/admin/ProductForm.tsx` `apps/api/src/modules/products/service.ts`
Bulk import from CSV, with recipes	**Completed**	Two tables, Products and Recipes (bill of materials); dishes are matched by name and a new dish needs its recipe. Every row is checked first (dry run); import is enabled only when no row has an error.	`apps/web/src/app/admin/import/page.tsx` `apps/api/src/modules/products/bulkImport.ts`
Downloadable import template	**Completed**	CSV templates for each table and a two-sheet Excel template with drop-down lists of categories and raw materials.	`apps/api/src/modules/products/routes.ts`
Bulk import from Excel files	**Completed**	The .xlsx file is converted to CSV and goes through the same checks.	`apps/api/src/modules/products/xlsxImport.ts`
Barcode entry by scanner or typing	**Removed from scope**	Removed with barcode lookup at the panel's request (November 2026).	—
Searchable and filterable admin catalogue	**Partial**	Search by name or category and a category filter; no price-range or popularity filter.	`apps/web/src/app/admin/products/page.tsx`

Feature	Status	Remarks	Main files
Add or edit categories	**Completed**	Administrator page: add, rename, emoji icon, NEW badge, reorder the kiosk's category rail, delete when empty; kiosks update live.	`apps/web/src/app/admin/categories/page.tsx` `apps/api/src/modules/categories/service.ts`
7. Inventory monitoring and stock adjustments			
Live stock dashboard with level indicators	**Completed**	Key figures, "X items at or below minimum stock" with See more, and Needs restock and Out of stock tiles that open the raw materials already filtered (panel feedback).	`apps/web/src/app/admin/page.tsx`
Minimum stock level per item	**Completed**	For raw materials and ready-made items.	`apps/web/src/components/admin/ProductForm.tsx` `apps/web/src/components/admin/RawMaterialForm.tsx`
Low-stock list and banner	**Completed**	Dashboard and Raw materials page (the separate Inventory page was merged into it).	`apps/web/src/app/admin/raw-materials/page.tsx` `apps/api/src/modules/inventory/routes.ts`
Automated low-stock notifications by e-mail	**Completed**	A database trigger queues an alert and the worker e-mails it through Brevo (HTTPS, as Railway blocks SMTP). Switched on in production on 30 October 2026.	`apps/api/prisma/migrations/20261013160000_low_stock_alerts/migration.sql` `apps/worker/src/jobs/sendLowStockAlerts.ts` `apps/worker/src/lib/mailer.ts`
Stock adjustments with a required reason	**Completed**	Damaged goods and spoilage can only lower stock.	`apps/web/src/components/admin/AdjustStockModal.tsx` `apps/web/src/components/admin/RawMaterialAdjustModal.tsx` `apps/api/src/modules/inventory/service.ts`
Adjustment log (who, when, before/after, reason)	**Completed**	One Stock movements log for raw materials and ready-made items: sales, restocks, adjustments and cancelled orders put back, filtered by item, kind of change and dates.	`apps/web/src/app/admin/raw-materials/page.tsx` `apps/api/src/modules/inventory/movements.ts`
Raw-material search and categories	**Completed**	Added from the panel's review: a search bar and a category filter; each raw material has a category from a fixed list, set in its form.	`apps/web/src/app/admin/raw-materials/page.tsx` `apps/web/src/lib/rawMaterialCategories.ts`
Remove a raw material	**Completed**	Added by the proponents: Remove item in the raw material's form, after a confirmation that its movement history goes with it; refused, naming the dishes, while a recipe still uses it.	`apps/web/src/components/admin/RawMaterialForm.tsx` `apps/api/src/modules/raw-materials/service.ts`
8. Analytics, reports and data export			
Daily, weekly and monthly sales summaries	**Completed**	Last 14 days, 8 weeks or 6 months.	`apps/api/src/modules/reports/service.ts` `apps/web/src/app/admin/analytics/page.tsx`

Feature	Status	Remarks	Main files
Top-selling products	**Completed**	All-time ranking.	`apps/api/src/modules/reports/service.ts`
Inventory movement	**Completed**	Units sold against current stock.	`apps/api/src/modules/reports/service.ts`
Profitability per item	**Completed**	Cost, price, margin and gross profit.	`apps/api/src/modules/reports/service.ts`
Revenue, transaction count and average order value	**Completed**	Average order value = revenue ÷ paid orders, per period and in the sales export.	`apps/web/src/app/admin/analytics/page.tsx` `apps/api/src/modules/reports/metrics.ts`
CSV export	**Completed**	Every report.	`apps/api/src/modules/reports/export.ts`
PDF and Excel export	**Completed**	Every report.	`apps/api/src/modules/reports/export.ts`
9. Authentication, roles and user management			
Staff and administrator sign-in	**Completed**	15-minute access token renewed in the background through the admin site's own address; signed out after 6 hours without use (the client's rule, 3 November 2026).	`apps/api/src/modules/auth/service.ts` `apps/api/src/lib/jwt.ts` `apps/web/src/app/login/page.tsx`
Anonymous kiosk session for customers	**Completed**	2-hour token, renewed automatically.	`apps/api/src/modules/auth/service.ts` `apps/web/src/lib/auth.ts`
Role-based access control (Administrator, Staff, Customer)	**Completed**	Checked on the server for every API route and on the pages.	`apps/api/src/middleware/authorize.ts` `apps/web/src/components/admin/AdminRouteGuard.tsx`
User administration	**Completed**	Create, change role, suspend or reactivate; an administrator cannot demote or suspend their own account.	`apps/web/src/app/admin/users/page.tsx` `apps/api/src/modules/users/service.ts`
Signed-out visitors kept out of staff pages	**Completed**	Redirect to sign-in; staff pages do not exist on the customer link.	`apps/web/src/components/StaffGuard.tsx` `apps/web/src/proxy.ts`
10. Security measures			
Encrypted transport (HTTPS)	**Completed**	All deployed services use HTTPS addresses on Railway.	—
SQL-injection prevention	**Completed**	Prisma parameterised queries; the only raw queries are in the backup restore check and use internal table names.	`apps/worker/src/jobs/restoreCheck.ts`
Password hashing and token security	**Completed**	bcrypt; refresh token in an httpOnly cookie.	`apps/api/src/modules/users/service.ts` `apps/api/src/modules/auth/routes.ts`
Emailed sign-in code	**Completed**	After the password, a 6-digit code is emailed through Brevo; it lasts 10 minutes, allows 5 tries and works once. Switched on in production on 30 October 2026.	`apps/api/src/modules/auth/loginCodes.ts` `apps/api/src/lib/loginMail.ts`

Feature	Status	Remarks	Main files
Input validation layer	**Completed**	Zod schemas on every module.	`apps/api/src/modules/orders/schema.ts` `apps/api/src/modules/products/schema.ts`
Security headers and CORS	**Completed**	helmet; only the kiosk and admin addresses may call the API.	`apps/api/src/app.ts` `apps/api/src/lib/origins.ts`
Ownership checks and signed payment webhooks	**Completed**	A kiosk session can read only its own orders; webhook signatures are verified.	`apps/api/src/modules/orders/routes.ts` `apps/api/src/lib/paymongo.ts`
Rate limiting	**Completed**	Sign-in, session renewal, order placement, kiosk sessions and payment start.	`apps/api/src/middleware/rateLimit.ts`
11. Background worker and backups			
Hourly low-stock check	**Completed**	Logs items at or below their minimum.	`apps/worker/src/jobs/lowStockSweep.ts`
Scheduled database backup	**Completed**	Nightly at 11:00 PM (Asia/Manila) with pg_dump.	`apps/worker/src/jobs/backupDatabase.ts` `apps/worker/src/index.ts`
Off-site backup copy	**Completed**	Uploaded to Cloudflare R2; a restore check job is provided.	`apps/worker/src/lib/storage.ts` `apps/worker/src/jobs/restoreCheck.ts`
Offline transaction queuing and synchronisation	**Removed from scope**	Removed from the manuscript by adviser decision.	—
12. Offline support (PWA)			
Installable Progressive Web App	**Removed from scope**	Removed from the manuscript by adviser decision.	—
Offline caching of the menu	**Removed from scope**	Removed from the manuscript by adviser decision.	—
Offline order queue with automatic synchronisation	**Removed from scope**	Removed from the manuscript by adviser decision.	—
13. Hardware integration			
Barcode scanner input	**Removed from scope**	Removed with barcode lookup at the panel's request (November 2026).	—
Thermal printers (receipt and kitchen)	**Completed**	Two RP21UB-Y 58 mm USB printers, one at the kiosk and one at the kitchen display, each driven by the print service in the repository; text is cleaned of printer control characters and only the system's own sites may print. Both verified on 27 October 2026.	`print-service/server.js` `print-service/README.md`

Feature	Status	Remarks	Main files
14. QR code and multi-device access			
Kiosk usable on phones, tablets and desktops	**Completed**	Responsive layout; the back-office sidebar stays visible at every width.	`apps/web/src/app/page.tsx` `apps/web/src/components/admin/AdminShell.tsx`
QR code that opens the kiosk	**Completed**	Generated in the browser; printable poster and PNG download.	`apps/web/src/app/admin/kiosk-qr/page.tsx`
15. Deployment			
Web application and API on Railway	**Completed**	Separate customer (kiosk) and staff (admin) addresses.	`apps/web/src/proxy.ts`
Managed PostgreSQL on Railway	**Completed**	Migrations applied on API start.	`apps/api/prisma/schema.prisma`
Worker running in production	**Completed**	Builds from its own Dockerfile; cloud backups verified on 15 October 2026.	`apps/worker/Dockerfile`
16. Testing and quality assurance			
Automated unit tests	**Completed**	18 test files.	`apps/api/test/unit`
Automated integration and security tests	**Completed**	32 test files, including an access matrix for every API route.	`apps/api/test/integration`
Type-checking	**Completed**	No type errors in the API, web application or worker.	—
Linting	**Completed**	ESLint reports no errors or warnings in the web application (resolved on 30 October 2026).	—
Documentation of test sessions	**Completed**	The testing document (Automated-Testing-Results) records the suite of 762 test cases over eighteen sessions. The seventeenth, on 13 November 2026, added 66 for the bill of materials and retired 42 whose features were removed or rewritten (barcode lookup, dishes counted in units, the single-table import); the eighteenth, the same day, added 25 for the counter's point of sale, the stock movements log and raw-material categories.	—
Usability evaluation (ISO/IEC 25010)	**Not yet started**	Questionnaire prepared and approved by the adviser; not yet administered.	—
Structured testing on the physical kiosk	**Not yet started**	Planned. Receipt and kitchen-ticket printing were verified on the physical printers on 27 October 2026, but no structured test has been run and recorded (ordering, printing, idle reset, touch use).	—

5. Completed Features

The completed work covers the full ordering cycle and the back-office tools around it. The figures are low-fidelity wireframes drawn from the current code (`docs/diagrams/`); each names the files it is based on.

5.1 Customer ordering

The kiosk opens on a welcome screen and asks once whether the customer will dine in or take out. The menu shows one category at a time, with a category rail, keyword search and price-range chips; sold-out dishes are dimmed and update live. Customers add dishes directly or through an item sheet with quantity, quick instruction options and a note. Checkout shows the order, an estimated preparation time based on the kitchen queue, and the payment choice (Pay at counter, or QR Ph once online payment is set up). Placing the order returns a receipt with a unique order number; with QR Ph the kiosk first shows the payment QR code to scan. Inactivity triggers a warning and then resets the kiosk for the next customer.

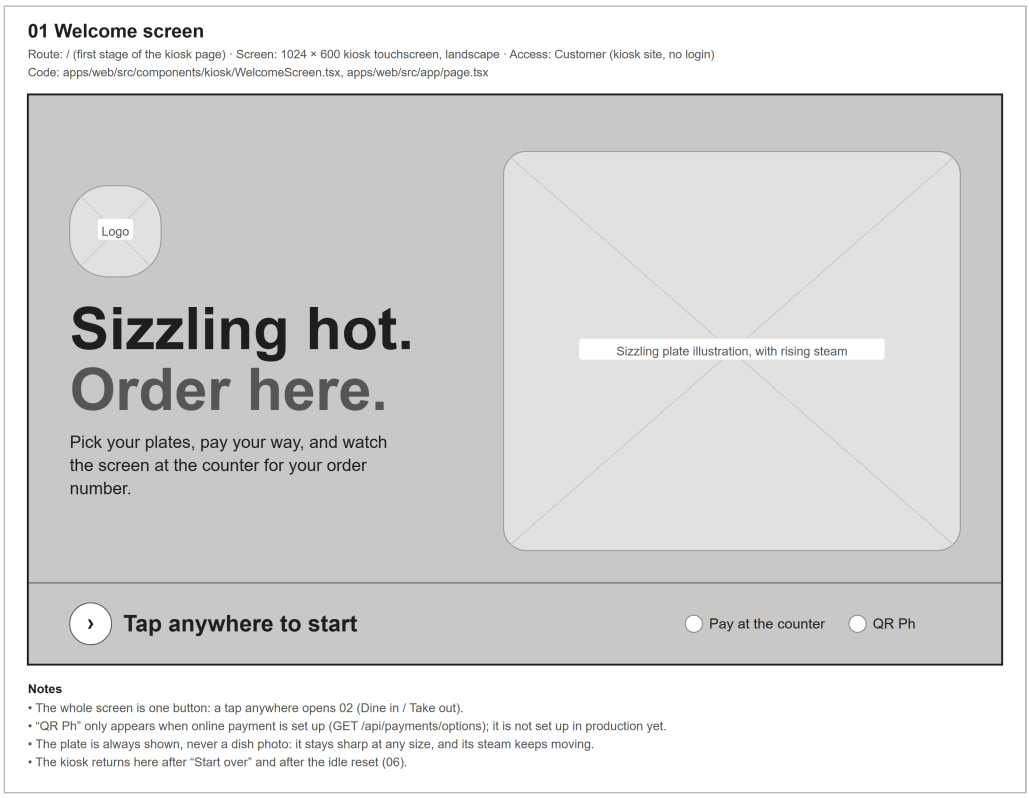


Figure 2. Kiosk welcome screen (wireframe).

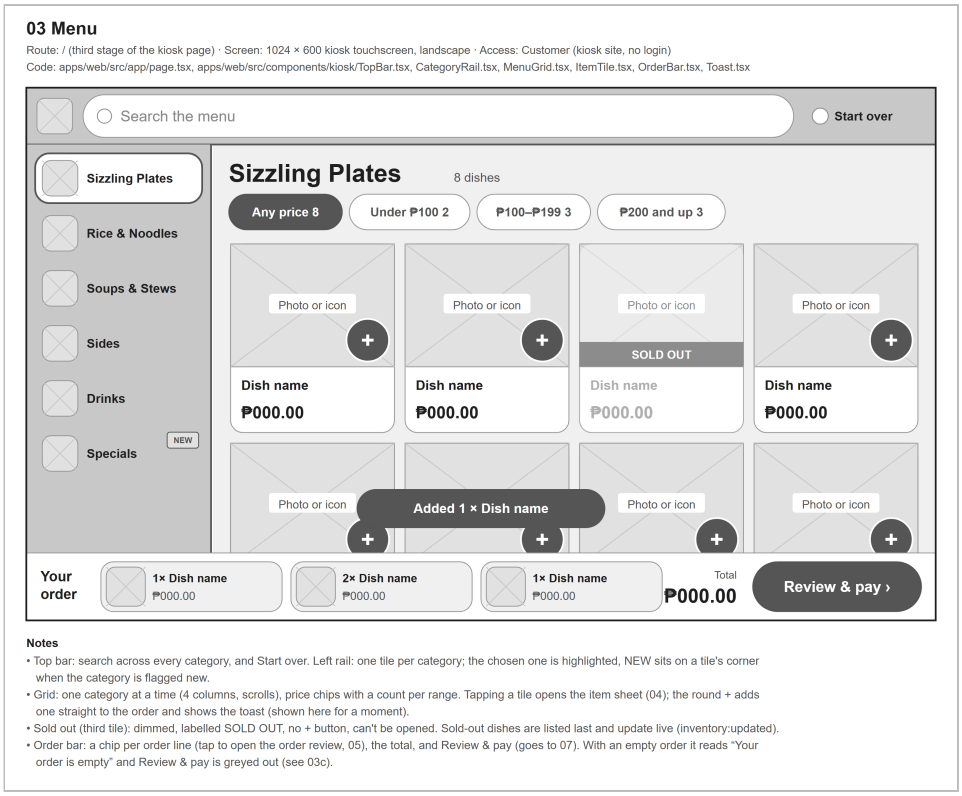


Figure 3. Kiosk menu with category rail, dish grid, price filter and order bar (wireframe).

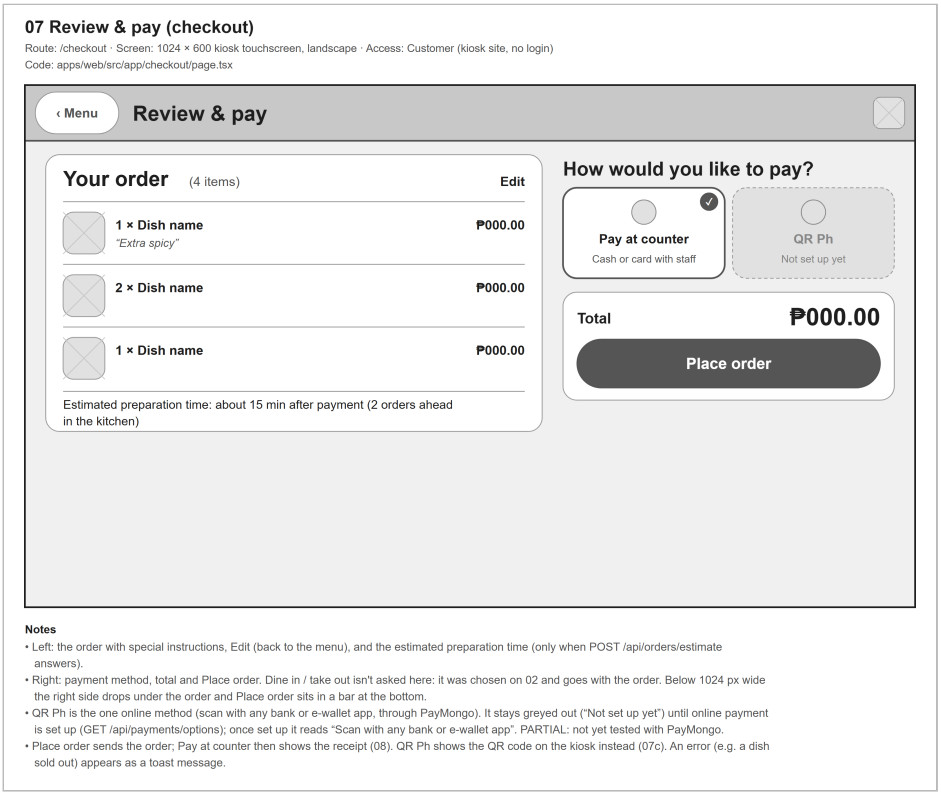


Figure 4. Review and pay screen (wireframe).

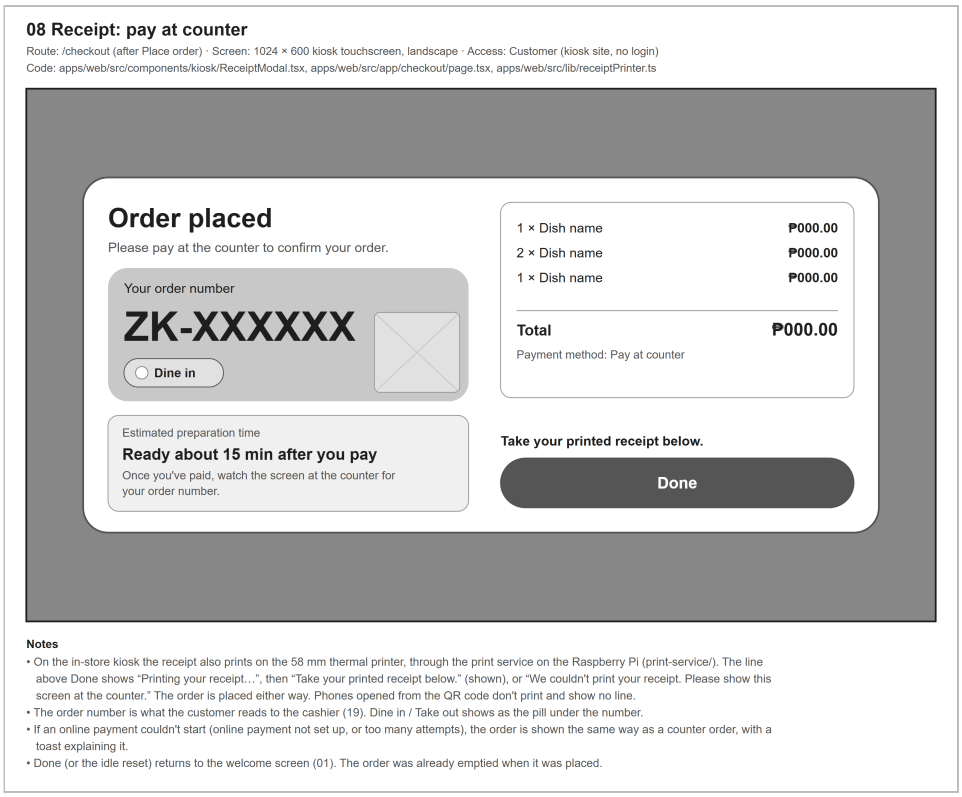


Figure 5. Receipt after a counter order, with the print status (wireframe).

5.2 Counter payment and kitchen

Orders awaiting payment is the counter's light point of sale, with three tabs. In the Queue, the cashier finds the order by its number and takes payment: cash, with the change worked out, or GCash or Maya, recorded with the reference number from the customer's phone. Taking payment marks the order paid, deducts the stock and creates kitchen tasks in a single database transaction; if any item has run out, nothing changes and the cashier is told before the money is kept. A receipt then shows on screen and prints on the counter computer's printer. New order rings up a walk-in customer from a menu grid, and Paid orders lists the orders in the kitchen, which the cashier can cancel with a reason. Unpaid orders can be cancelled in the Queue. Paid orders appear immediately on the kitchen display, where each dish moves from Pending to In progress to Completed, with dine-in and take-out orders distinguished. As an order appears, a ticket with its number and dishes prints on the kitchen printer. The order itself becomes Preparing when its first dish starts and Ready when the last is done; a status board on the screen at the counter then shows its number under Ready for pickup, until the kitchen presses Handed over. The payment queue updates by itself as orders are placed and paid.

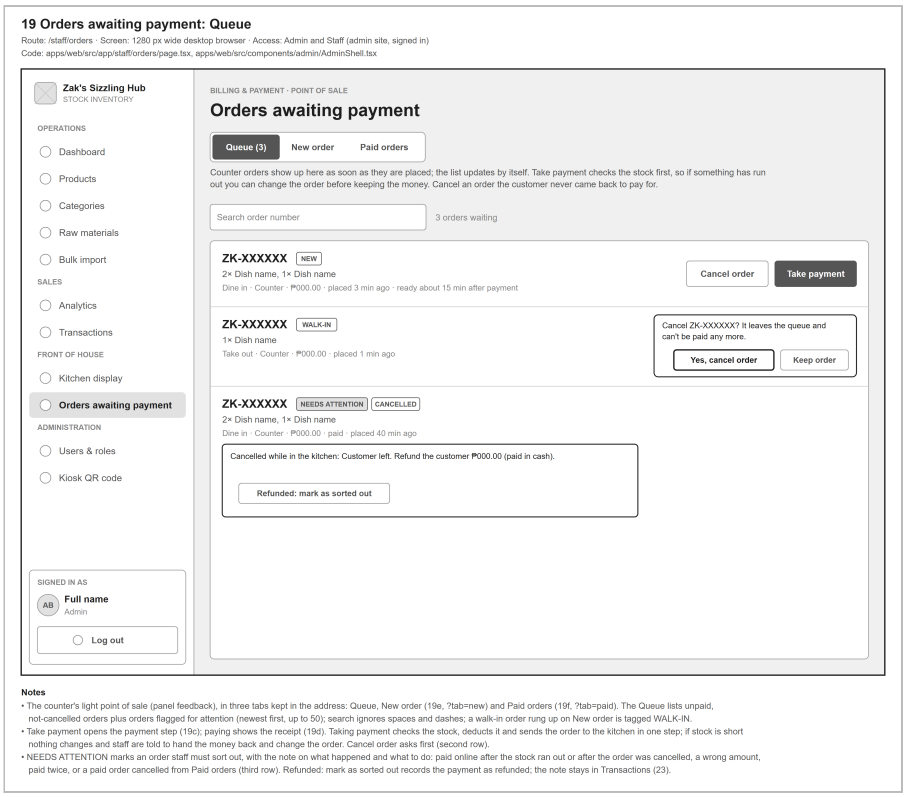


Figure 6. Orders awaiting payment, used by the cashier (wireframe).

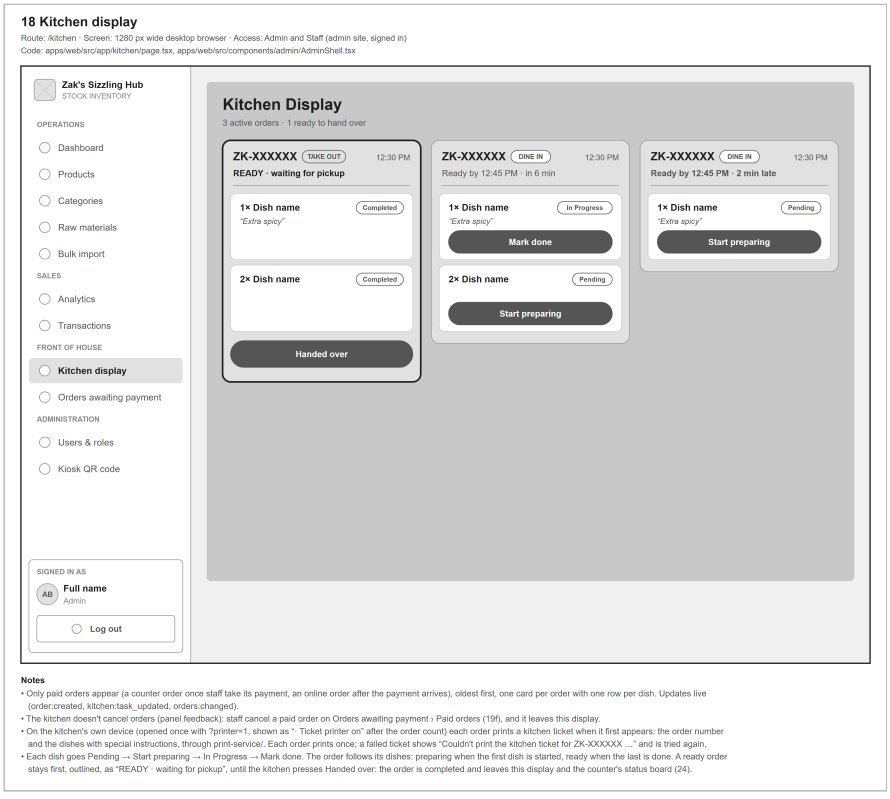


Figure 7. Kitchen display (wireframe).

5.3 Inventory

Following the panel's review, every dish is made from a bill of materials: its recipe of raw materials is required, and the kiosk shows how many servings the raw materials still allow. Items sold as they are, such as

bottled drinks, are kept in a built-in Ready-made category and counted in pieces. Each raw material and ready-made item has a minimum level; the dashboard says how many are at or below it, with links to the Raw materials page already filtered, and each is queued for an e-mail alert. The Raw materials page has a search bar, a category filter, the servings of each dish, the ready-made items, and one Stock movements log of every change. The Administrator can remove a raw material after a confirmation, unless a recipe still uses it. Every manual change requires a reason and is logged with the person, the time and the before and after quantities. Barcode lookup was removed at the panel's request.

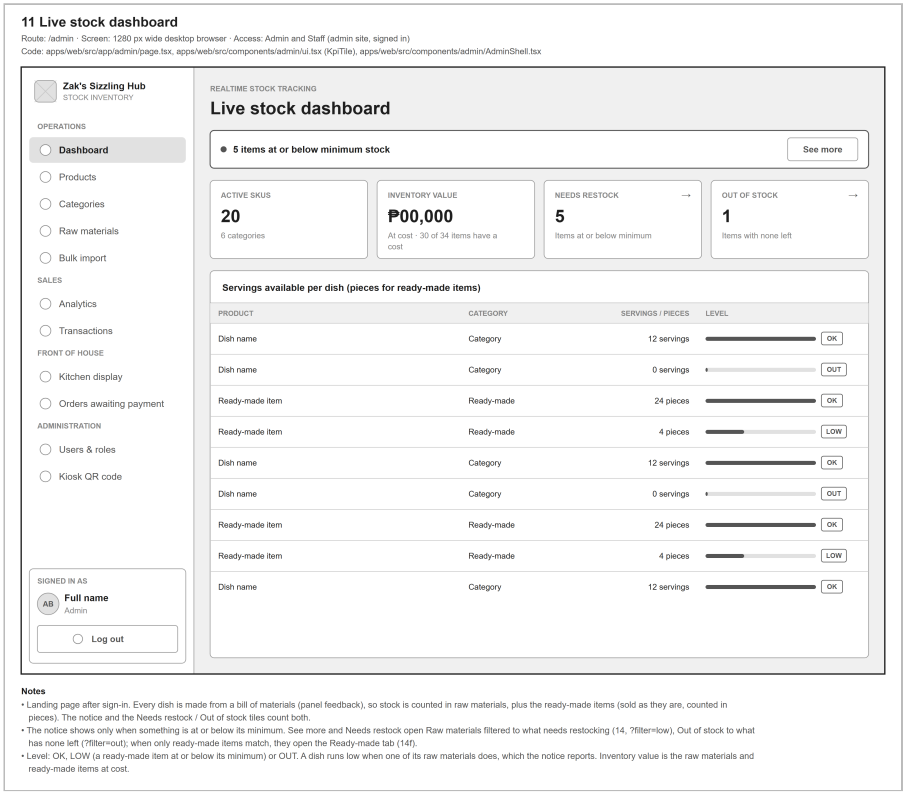


Figure 8. Live stock dashboard (wireframe).

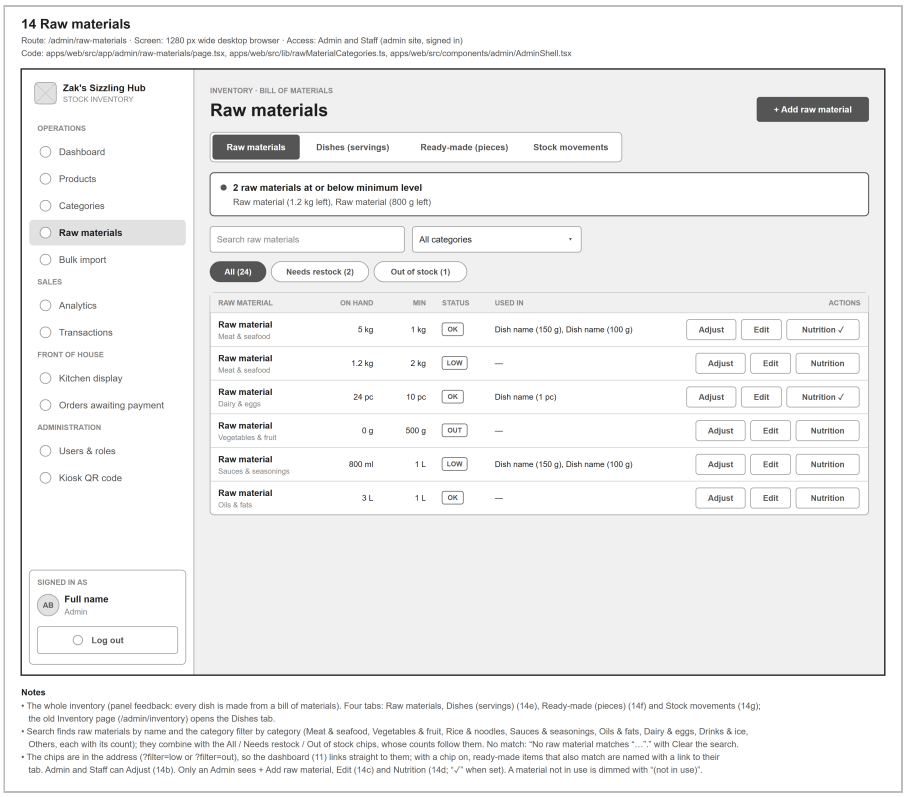


Figure 9. Raw-material stock and movement log (wireframe).

5.4 Products, analytics, users and operations

The Administrator creates and edits products with validation, photos, preparation times, recipes, ingredients, allergens and nutrition (looked up in USDA FoodData Central or calculated from the recipe), manages the menu's categories, sets each dish's special-instruction options, and imports products and their recipes in bulk from CSV or Excel after a row-by-row check. Analytics show daily, weekly and monthly sales with the average order value, top-selling products, profitability and inventory movement, each exportable as CSV, Excel or PDF, and a transaction history lists every order with its timeline. User accounts can be created, assigned a role, suspended and reactivated. The Administrator also prints a QR code that opens the kiosk on customers' phones. In operation, the worker backs up the database every night to off-site storage, and separate web addresses keep staff pages off the customer link.

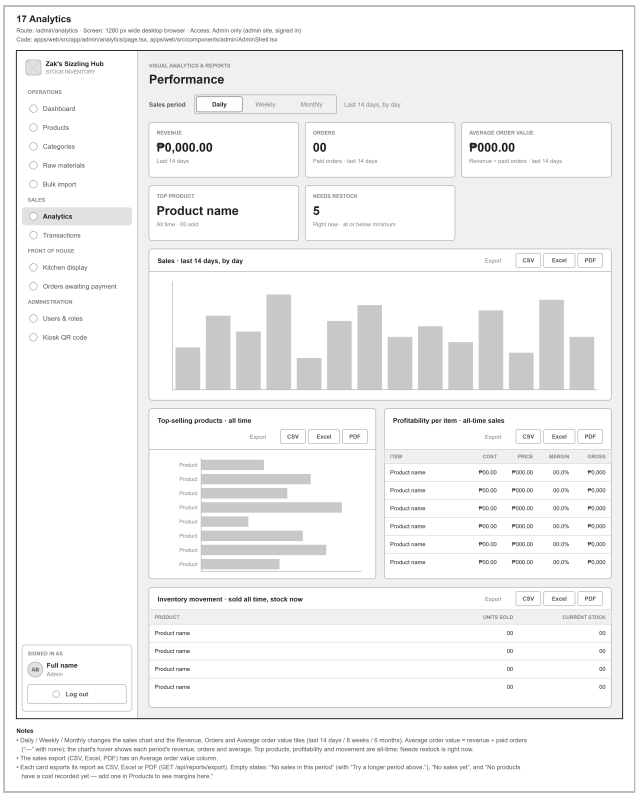


Figure 10. Analytics and report export (wireframe).

5.5 Security and quality

Passwords are hashed with bcrypt, staff and administrators confirm each sign-in with a code emailed to them, staff sessions use short-lived tokens, all input is validated, responses carry security headers, only the kiosk and admin addresses may call the API, payment webhooks are verified by signature, and repeated sign-in, order and payment attempts are rate limited. The API and worker are covered by automated unit, integration and security tests (Section 8).

6. Partial and Pending Features

The table lists every feature that is Partial or Not yet started, with what exists and what remains.

Feature	Status	What is done and what is left
Filter by price range or popularity 1. Kiosk ordering interface	Partial	Price-range filter implemented; no popularity filter.
Online payment through PayMongo (QR Ph) 2. Checkout and payments	Partial	QR Ph is the one online method; it replaced GCash and Maya on 26 October 2026. The kiosk shows the QR Ph code on its own screen, checks for the payment every 3 seconds, and also accepts PayMongo's signed webhook; covered by automated tests with PayMongo simulated. Not yet run with PayMongo keys; production has none yet.
QR-code payment option (manuscript wireframe) 2. Checkout and payments	Not yet started	Not present in the code.
Testing of PayMongo with sandbox and live keys 2. Checkout and payments	Not yet started	Planned. Requires PayMongo keys and the client's approval.
Searchable and filterable admin catalogue 6. Product and catalogue management	Partial	Search by name or category and a category filter; no price-range or popularity filter.
Usability evaluation (ISO/IEC 25010) 16. Testing and quality assurance	Not yet started	Questionnaire prepared and approved by the adviser; not yet administered.
Structured testing on the physical kiosk 16. Testing and quality assurance	Not yet started	Planned. Receipt and kitchen-ticket printing were verified on the physical printers on 27 October 2026, but no structured test has been run and recorded (ordering, printing, idle reset, touch use).

Removed from scope

Offline transaction queuing, the installable Progressive Web App, offline caching of the menu, and the offline order queue were removed from the manuscript by the adviser's decision (`docs/manuscript-fixes/CHANGES.md`, section 1). The system requires an internet connection. Barcode lookup, with barcode entry by scanner or typing, was removed in November 2026 at the panel's request, since every dish's stock now comes from its bill of materials.

7. Known Issues and Limitations

7.1 Functional

1. **Ready notice on customers' phones.** The status board on the counter screen tells customers their order is ready; a customer who ordered on their own phone through the QR code is not notified on the phone, whose receipt does not update by itself. Left for future development.
2. **Changing an order.** When stock runs out at confirmation, the cashier is told to change the order with the customer, but no screen edits an existing order. The API provides routes to change or remove order items that the web application does not use; in practice the order is cancelled and placed again.
3. **Nutrition values are estimates.** They are looked up in USDA FoodData Central (the key is set on the production API). Values calculated from a recipe use raw ingredients and do not account for cooking.
4. **Online payment not yet verified.** The QR Ph flow (code on the kiosk, status check, webhook) has only been tested with PayMongo simulated; two details depend on PayMongo's live behaviour (attaching the QR Ph method with the secret key, and the location of the test-mode simulate link). The return page of the earlier hosted-checkout flow (`/checkout/success`) remains in the code but is no longer reached.
5. **Kitchen printing needs a browser on the printer's device.** The kitchen ticket is sent by the kitchen display open (in printer mode) on the device the kitchen printer is plugged into, a Raspberry Pi or Linux computer running the print service. Other devices, such as an Android tablet, can show the kitchen display but cannot print.
6. **Account passwords.** Users cannot change their own password, and temporary passwords given to new users are not required to be changed.

7.2 Security and maintenance

1. **Hosting provider incident.** On 21 October 2026 Railway disclosed unauthorised access to customer environment variables and advised rotating every secret. The three token secrets have been replaced; the database password and the backup-storage and e-mail keys remain to be rotated, and Railway paused deployments during its investigation.
2. **Dependency advisories.** `npm audit` reports 18 advisories in production dependencies (3 critical, 9 high, 6 moderate), all with fixes available. The critical advisory in Next.js concerns `next/og`, which the system does not use; the others should still be resolved by upgrading.
3. **Browser token storage.** Staff access tokens are kept in browser storage, which a cross-site scripting flaw could expose; the impact is limited by the 15-minute token lifetime and the httpOnly refresh cookie.
4. **No automated tests for the web interface.** The screens were checked manually in a browser.

8. Testing Status

Activity	Result	Details
Automated tests (API and worker)	**All passed**	Run on 14 November 2026: 768 test cases in 50 files; all 768 passed, 0 failed, including the backup tests (run with pg_dump 18). Runs with the test files in random order also pass (see Test documentation). No test is marked as a known defect.
Unit tests	18 files	Token handling, sign-in codes and their emails (Brevo), middleware, preparation-time formula, stock fulfilment, CSV and Excel import of products and recipes, report export and the average order value, e-mail content, sending e-mail through Gmail, Resend and Brevo, backup storage, allowed origins, the PayMongo QR Ph requests, nutrition (USDA mapping and calculation from a recipe).
Integration and security tests	32 files	Against a separate test database: an access matrix covering every API route and role, sign-in with emailed codes (simulated Brevo), orders, order status and the status board, the live payment queue, transaction history, categories, nutrition, stock, raw materials, their categories and removing them, the bill of materials and ready-made items, the counter's point of sale (cash, GCash and Maya payments, walk-in orders, paid orders), the stock movements log, payments including QR Ph (simulated gateway), rate limiting, malformed input, product images, imports, exports, low-stock alerts and backups.
Type-checking	No errors	API, web application and worker.
Linting	No errors	Web application: 0 errors, 0 warnings.

Activity	Result	Details
Test documentation	Complete	The testing document records the suite of 768 test cases over nineteen sessions; the ninth session (29 October 2026) added 59 for the sign-in codes (a weakness in requesting a new code, found afterwards by an automated security review, was fixed the same day) and the tenth (30 October 2026) 5 for sending low-stock e-mail through Brevo, all passed. The eleventh (31 October 2026) added 14 for online payments that need staff, after a code review found five defects, all fixed the same day. The twelfth (3 November 2026) added 4 for staying signed in on the admin side, after the client reported being signed out after a few minutes. The thirteenth (4 November 2026) added 17 for cancelling a paid order from the kitchen and recording the refund, and the fourteenth 11 after an automated security review of that change found defects in the refund records, the fifteenth 4 after a further review found rarer ones, and the sixteenth 4 for one more found while correcting those, all fixed the same day. After the panel's review, the seventeenth (13 November 2026) added 66 for the bill of materials and retired 42 cases whose features were removed or rewritten, and the eighteenth added 25 for the counter's point of sale, the stock movements log and raw-material categories; each of these two sessions also corrected a test file that failed only when the test files ran in a particular order. The nineteenth (14 November 2026) added 6 for removing a raw material.
Sign-in codes in production	Verified	Switched on 30 October 2026: test emails from the Users page arrived, and an administrator signed in on the live admin site with the emailed code.
Web interface	Manual only	Checked in a browser at the kiosk size (1024 × 600) and on desktop; no automated tests.
Physical kiosk	Partial	Receipt printing at the kiosk and ticket printing at the kitchen verified on both printers on 27 October 2026; no structured, recorded test yet (ordering, printing, idle reset, touch use, and the counter receipt on the counter computer's printer).
Online payment	Not yet started	QR Ph never run against PayMongo with sandbox or live keys.
Usability (ISO/IEC 25010)	Not yet started	Questionnaire prepared and approved by the adviser; not yet administered.

9. Next Steps

The remaining work is listed in the recommended order: security first, then the items the panel is most likely to ask to see, then evaluation, then functional gaps.

1. **Complete the secret rotation.** Replace the database password and the backup-storage and e-mail keys on Railway once deployments resume, then confirm that every service redeploys and that ordering, sign-in and backups still work.
2. **Replace the starting stock in production with real counts.** Every dish on the live menu has its recipe, and on 14 November 2026 each raw material in a recipe was stocked for 50 servings of every dish that uses it, with its minimum level at 10 servings' worth. Count the actual stock of the raw materials and ready-made items and enter it, and remove the test product and test raw material.
3. **Conduct structured testing on the physical kiosk and counter.** Ordering, printing, the idle reset, touch use at 1024 × 600, and printing a counter receipt from the point of sale; record the results.
4. **Test online payment (QR Ph).** With PayMongo test keys first, paying through the test-mode simulate link, then with live keys after the client's approval; register the `payment.paid` webhook, and check that the receipt prints as PAID and the kitchen ticket prints once the payment arrives.
5. **Administer the ISO/IEC 25010 evaluation** with the approved questionnaire and analyse the results.
6. **Set up the new features in production:** open the status board (`/board`) on the screen at the counter, and have the owner confirm each dish's ingredients, allergens and nutrition (entered on the live site on 28 October 2026 as estimates).
7. **Close the remaining functional gaps:** a page where Staff change their own password (administrators can now set any account's password and email in Users), and optionally a ready notice on customers' phones.
8. **Maintenance:** upgrade dependencies with published advisories.